

Nombre y Apellido: N° Legajo:

Recuperatorio Segundo Parcial de Programación Imperativa
29/06/2026

	Ejercicio 1	Ejercicio 2	Ejercicio 3	Nota
Calificación	/2	/4	/4	

- ★ **Condición mínima de aprobación: Sumar 5 (cinco) puntos.**
- ★ **Los ejercicios que no se ajusten estrictamente al enunciado, no serán aceptados.**
- ★ **No usar variables globales ni static.**
- ★ **No es necesario escribir los #include**
- ★ **Escribir en esta hoja Nombre, Apellido y Legajo**
- ★ **Antes de retirarse del aula confirmar con el docente que se haya recibido la entrega**

Ejercicio 1

Escribir una función **recursiva** **sumOK** que reciba como **único parámetro** un **vector de enteros** donde todos sus elementos son mayores o iguales a cero, con **-1 como marca de final**.

La función debe **retornar 1** si **cada tercer elemento es la suma de los dos anteriores**, y 0 en caso contrario.

```
int main(void) {
    {   int v[] = {1, 2, 3, 2, 1, 3, 4, 3, 7, 2, 0, 2, -1};
        assert(sumOK(v));   }
    {   int v[] = {1, 2, 3, 2, 1, -1}; // falta un elemento que sea 3
        assert(!sumOK(v));   }
    {   int v[] = {1, 2, 3, 2, 2, 5, 2, 4, 6, -1}; // 2+2 no es 5
        assert(!sumOK(v));   }
    {   int v[] = { -1};
        assert(sumOK(v));    }
    return 0;
}
```

NO SE ADMITIRÁ UNA SOLUCIÓN QUE TENGA UN CICLO DENTRO DE LA FUNCIÓN.
NO DEFINIR MACROS NI FUNCIONES AUXILIARES.

Ejercicio 2

Se implementará un TAD para almacenar **la oferta de materias en un cuatrimestre para las distintas carreras de una universidad**. Cada **carrera** se identifica con un código de hasta dos dígitos, por ejemplo **1** para Ingeniería Química, **11** para Medicina, **80** para Informática, **73** para Enfermería, etc.

Cada **materia** (también se lo puede llamar curso) se identifica con un código de hasta 5 dígitos: los dos primeros corresponden a la carrera y los 3 restantes a la materia en sí, por ejemplo **1020** es una materia de Ingeniería Química (de código 020), **11010** es

una materia de Medicina (de código 010), **80888** es una materia de Informática (de código 888).

Para ello se definió la siguiente interfaz:

```
typedef struct coursesCDT * coursesADT;

coursesADT newCoursesADT(void);

void freeCoursesADT(coursesADT courses); // NO IMPLEMENTAR

/**
 * Agrega una nueva carrera. Si el código es inválido
 * o si ya existe una carrera con ese código no cambia nada y retorna cero. Sino retorna 1
 */
int addDegree(coursesADT courses, unsigned degreeCode, const char * degreeName);

/**
 * Agrega un curso a una carrera. Si el código es inválido, o la carrera no existe
 * o ya existe el curso para esa carrera no cambia nada y retorna cero. Sino retorna 1
 */
int addCourse(coursesADT courses, unsigned courseCode, const char * courseName);

/**
 * Retorna la cantidad de cursos en una carrera, o -1 si el código de carrera es inválido
 * o no existe
 */
int coursesCount(const coursesADT courses, unsigned degreeCode);

/**
 * Retorna una copia del nombre del curso, o NULL si no existe
 */
char * courseName(const coursesADT courses, unsigned courseCode);
```

Se pide implementar el TAD COMPLETO excepto la función freeCoursesADT, de forma tal que todas las funciones sean lo más eficiente posible.

```
int main(void) {
    coursesADT c = newCoursesADT();

    assert(coursesCount(c, 1) == -1);
    assert(courseName(c, 1020) == NULL);

    assert(addDegree(c, 1, "Ingenieria Quimica") == 1);
    assert(addDegree(c, 11, "Medicina") == 1);
    assert(addDegree(c, 80, "Astrologia") == 1);
    assert(addDegree(c, 180, "Reflexologia") == 0); // código invalido
    assert(addDegree(c, 100, "Mecanica") == 0); // código inválido
    assert(addDegree(c, 1, "Otro nombre") == 0); // código repetido

    assert(coursesCount(c, 1) == 0);
    assert(coursesCount(c, 11) == 0);
    assert(coursesCount(c, 80) == 0);

    assert(addCourse(c, 73001, "Enfermeria I") == 0); // No existe carrera 73

    assert(addCourse(c, 1020, "Quimica I") == 1); // Carrera: 1, curso: 20
    assert(addCourse(c, 1030, "Fisica I") == 1); // Carrera: 1, curso: 30
    assert(addCourse(c, 111234, "Extra") == 0); // código invalido
```

```

assert(addCourse(c, 11010, "Anatomia") == 1); // Carrera: 11, curso: 10
assert(addCourse(c, 11120, "Bioquimica") == 1); // Carrera: 11, curso: 120

assert(addCourse(c, 80001, "Constelaciones") == 1);
assert(addCourse(c, 80888, "Ascendentes I") == 1);

assert(addCourse(c, 1020, "Codigo repetido") == 0);

assert(coursesCount(c, 1) == 2);
assert(coursesCount(c, 11) == 2);
assert(coursesCount(c, 80) == 2);
assert(coursesCount(c, 73) == -1); // No existe

/* Consulta de nombres */
char * name;

name = courseName(c, 1020);
assert(strcmp(name, "Quimica I") == 0);
free(name);

name = courseName(c, 11120);
assert(strcmp(name, "Bioquimica") == 0);
free(name);

name = courseName(c, 80888);
assert(strcmp(name, "Ascendentes I") == 0);
free(name);

assert(courseName(c, 80002) == NULL);
assert(courseName(c, 73001) == NULL);

// Los nombres de carreras y de cursos se pueden repetir
assert(addDegree(c, 55, "Medicina") == 1);
assert(addCourse(c, 1080, "Quimica I") == 1);
assert(addCourse(c, 1081, "Anatomia") == 1);

freeCoursesADT(c);

return 0;
}

```

Ejercicio 3

Se desea implementar un TAD para dar soporte a un **poeta**, que tiene la particularidad de **escribir los versos en un orden arbitrario**.

Se asume que **cada verso** (un verso es una línea de un poema) **es un string corto, incluso puede ser vacío**. Pero el poema completo puede contener unos pocos o **miles de versos**.

Para más detalles sobre el funcionamiento ver el programa de test. Recordar que en C se concatenan los strings:

Esto	es equivalente a esto
"uno" "dos"	"unodos"
"primera\n" "segunda"	"primera\nsegunda"

Para ello se definió la siguiente interfaz:

```

typedef struct poemCDT * poemADT;

poemADT newPoemADT(void);

/**
 * Agrega un verso en la línea lineNumber (0 si es la primera línea)
 * Si en esa línea ya había un verso no vacío, desplaza todas las líneas que estaban
 * a partir de lineNumber una línea
 */
void addVerse(poemADT poem, const char * verse, size_t lineNumber);

/**
 * Reemplaza el verso de la línea lineNumber por el recibido
 * Si no había verso en esa línea, lo agrega
 */
void replaceVerse(poemADT poem, const char * verse, size_t lineNumber);

/**
 * Retorna el poema completo, desde el primero hasta el último verso
 * (incluyendo los vacíos comprendidos entre ellos),
 * agregando \n al final de cada uno
 */
char * poem(const poemADT poem);

void freePoem(poemADT poem);

```

Se pide **implementar el TAD completo**

Programa de prueba:

```

static void checkPoem(poemADT adt, const char * expected) {
    char * p = poem(adt);
    assert(strcmp(p, expected) == 0);
    free(p);
}

int main(void) {
    poemADT p = newPoemADT();
    char aux[100];

    /* Poema vacío */
    checkPoem(p, "");

    /* Primer verso */
    strcpy(aux, "En una noche oscura");
    addVerse(p, aux, 0);
    checkPoem(p, "En una noche oscura\n");

    /* Agregar al final. */
    strcpy(aux, "con ansias en amores inflamada");
    addVerse(p, aux, 1);
    checkPoem(p,
        "En una noche oscura\n"
        "con ansias en amores inflamada\n");

    /* Insertar al principio: desplaza todo */
    addVerse(p, "POEMA", 0);
    checkPoem(p,
        "POEMA\n"
        "En una noche oscura\n"
        "con ansias en amores inflamada\n");
}

```

```

/* Insertar en el medio: desplaza */
addVerse(p, "verso intermedio", 2);
checkPoem(p,
    "POEMA\n"
    "En una noche oscura\n"
    "verso intermedio\n"
    "con ansias en amores inflamada\n");

/* Reemplazar */
replaceVerse(p, "verso corregido", 2);
checkPoem(p,
    "POEMA\n"
    "En una noche oscura\n"
    "verso corregido\n"
    "con ansias en amores inflamada\n");

freePoem(p);

/* ----- */
/* Casos con líneas vacías automáticas */
/* ----- */

p = newPoemADT();

addVerse(p, "The end", 4);

checkPoem(p,
    "\n"
    "\n"
    "\n"
    "\n"
    "The end\n");

/* Inserción al principio, donde hay verso vacío: no desplaza */
addVerse(p, "Inicio", 0);

checkPoem(p,
    "Inicio\n"
    "\n"
    "\n"
    "\n"
    "The end\n");

strcpy(aux, "Linea 3");
addVerse(p, aux, 3); // Sobre línea vacía: no desplaza
strcpy(aux, "En la peor de las noches");
replaceVerse(p, aux, 2); // Sobre línea vacía
checkPoem(p,
    "Inicio\n"
    "\n"
    "En la peor de las noches\n"
    "Linea 3\n"
    "The end\n");

freePoem(p);

puts("OK");
return 0;
}

```